
Clase 079 — SVM para regresión (SVR)

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 5. Duración estimada: 50 min.

Clase 079 — SVM para regresión (SVR)

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 5. Duración estimada: 50 min.

Objetivo

Aplicar Support Vector Machines al problema de regresión: entender el truco del epsilon-insensitive loss (ajustar dentro de un tubo de tolerancia en vez de minimizar el error puntual), entrenar LinearSVR y SVR con kernel, y elegir cuándo conviene SVR sobre regresión lineal clásica.

Resultados de aprendizaje

Al finalizar la clase, vas a poder:

- Explicar la lógica de SVR: maximizar el ancho del tubo ϵ mientras se contienen la mayoría de los puntos dentro.
- Entrenar un LinearSVR y un SVR(kernel="rbf") de scikit-learn con sus hiperparámetros (epsilon, C, gamma).
- Interpretar el efecto de epsilon (ancho del tubo) y C (penalización por puntos fuera del tubo) en el bias-variance trade-off.
- Comparar SVR vs LinearRegression / Ridge en un dataset con outliers.
- Justificar cuándo SVR escala mal (SVR es $O(m^2-m^3)$) y conviene LinearSVR u otro modelo.

Temas

1. De SVM clasificación a SVM regresión: invertir el objetivo.
2. Epsilon-insensitive loss: errores menores a ϵ no se penalizan.
3. LinearSVR: caso lineal, escala bien ($O(m)$).
4. SVR con kernel RBF: regresión no lineal, costo cuadrático.
5. Hiperparámetros clave: epsilon, C, gamma, kernel.
6. Robustez frente a outliers comparada con OLS.

Definiciones y características

- SVR (Support Vector Regression): algoritmo de regresión que busca una función que se desvíe a lo sumo ϵ de cada target, manteniéndola lo más "plana" posible (margen máximo).
- Epsilon-tube (tubo ϵ): banda de ancho 2ϵ alrededor de la predicción donde los errores no cuentan. Solo los puntos fuera del tubo aportan a la pérdida.
- LinearSVR: implementación lineal de SVR basada en liblinear. Escala linealmente con el tamaño del dataset; no soporta kernels.
- Kernel SVR: SVR de scikit-learn usa libsvm; soporta kernels (rbf, poly, sigmoid) para capturar no linealidad, pero su complejidad es entre $O(m^2)$ y $O(m^3)$.
- Hiperparámetro C: controla la penalización por puntos fuera del tubo. C chico $\rightarrow$ modelo más regularizado (tubo "flexible"); C grande $\rightarrow$ ajuste más estricto.
- Hiperparámetro epsilon: define el ancho del tubo de insensibilidad. ϵ grande $\rightarrow$ menos vectores de soporte, modelo más simple; ϵ chico $\rightarrow$ ajuste más fino, más vectores.

- Robust regression: SVR es menos sensible que OLS a outliers porque la pérdida es lineal fuera del tubo (no cuadrática).

Dataset / recursos

- California Housing (`sklearn.datasets.fetch_california_housing`) para regresión realista.
- Dataset sintético con outliers (`make_regression` + ruido pesado) para mostrar robustez.
- Géron, Hands-On ML (3ª ed.), cap. 5, sección "SVM Regression".

Ejercicios

1. Tubo ϵ en 2D. Generá $y = 0.5x + \text{ruido}$, entrená `LinearSVR(epsilon=0.5)` y graficá la recta junto al tubo $\pm\epsilon$. Marcá los vectores de soporte (puntos fuera del tubo).
2. Efecto de epsilon. Repetí el ejercicio 1 con $\epsilon \in \{0.1, 0.5, 1.5\}$. ¿Cómo cambia la cantidad de vectores de soporte y el MSE en test?
3. Kernel RBF. En un dataset no lineal ($y = \sin(x) + \text{ruido}$), compará `LinearSVR` vs `SVR(kernel="rbf", gamma="scale")`. Reportá MAE y graficá ambas curvas.
4. Grid search. Sobre California Housing, hacé `GridSearchCV` con SVR variando $C \in \{0.1, 1, 10\}$ y $\gamma \in \{\text{"scale"}, 0.01, 0.1\}$. No uses más de 5 000 muestras (cuidado con el costo cuadrático).
5. Robustez vs OLS. Inyectá 5% de outliers en un dataset lineal y compará `LinearRegression`, `Ridge` y `LinearSVR`. ¿Cuál degrada menos?

Homework verifiable

Entregar un script `svr_california.py` que:

1. Cargue California Housing y aplique `StandardScaler` (¡SVR exige escalado!).
2. Entrene `LinearSVR(epsilon=0.5, C=1.0, random_state=42)` y un `SVR(kernel="rbf", C=10, gamma="scale")` sobre un subset de 5 000 muestras.
3. Reporte RMSE en test para ambos modelos.

Criterio de aceptación: `LinearSVR` RMSE < 0.85 y `SVR-RBF` RMSE < 0.65 sobre el split de test (`train_test_split(random_state=42, test_size=0.2)`).

Errores comunes

- No escalar los features. SVR es extremadamente sensible a la escala: sin `StandardScaler`, los hiperparámetros pierden sentido y el entrenamiento no converge.
- Usar SVR con kernel sobre datasets grandes. Para más de ~10 000 muestras, SVR se vuelve impracticable. Usá `LinearSVR` o `SGDRegressor(loss="epsilon_insensitive")`.
- Confundir epsilon de SVR con epsilon de optimizadores. Acá ϵ es el ancho del tubo de tolerancia, no una tolerancia numérica de convergencia.
- Dejar C en el default sin tunear. El default $C=1.0$ rara vez es óptimo; siempre validá con CV.
- Comparar tiempos sin `dual="auto"`. En `LinearSVR`, el parámetro `dual` cambia drásticamente el tiempo según n_{samples} vs n_{features} .

Preguntas frecuentes

- ¿SVR vs Linear Regression cuándo conviene cuál? OLS minimiza el error cuadrático medio (sensible a outliers, óptimo si el ruido es gaussiano). SVR ignora errores chicos (dentro del tubo ϵ) y penaliza linealmente los grandes: más robusto a outliers y suele generalizar mejor con pocos datos ruidosos. Si tu dataset es limpio y grande, OLS/Ridge suele ser más simple y rápido.
- ¿Cómo elijo epsilon? Empezá con una fracción del desvío estándar del target (p. ej. $\epsilon \approx 0.1 \cdot \sigma(y)$) y ajustá con CV. Un ϵ muy chico convierte SVR en algo cercano a regresión cuadrática; uno muy grande genera un modelo casi constante.
- ¿SVR devuelve probabilidades o intervalos? No. SVR predice un valor puntual. Para intervalos de predicción usá quantile regression, conformal prediction o un modelo bayesiano.
- ¿Por qué SVR con kernel es tan lento? Calcula la matriz kernel ($m \times m$) y resuelve un QP. La complejidad va de $O(m^2)$ a $O(m^3)$. Para datasets grandes, LinearSVR o Nystroem + LinearSVR (kernel approximation) son alternativas viables.
- ¿Conviene SVR para deep learning / problemas con millones de muestras? No. Para esas escalas, gradient boosting (XGBoost, LightGBM) o redes neuronales superan a SVR tanto en performance como en costo.

Referencias

- Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3ª ed.), O'Reilly, cap. 5 — "SVM Regression".
- Smola, A. & Schölkopf, B. A Tutorial on Support Vector Regression (2004).
- scikit-learn docs: `sklearn.svm.SVR` y `sklearn.svm.LinearSVR`.
- scikit-learn user guide: Support Vector Machines — Regression.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 080 — Árboles de decisión: entrenamiento, visualización, CART

Apéndice: notebook (primer bloque)

Generamos $y = 0.5 \cdot x + \text{ruido}$, entrenamos `LinearSVR(epsilon=0.5)` y dibujamos la recta con su tubo $\pm \epsilon$. Los puntos fuera del tubo son los vectores soporte.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import LinearSVR, SVR
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error

RND = 42
rng = np.random.default_rng(RND)
```

```
x = np.linspace(0, 5, 120)
y = 0.5 * x + rng.normal(0, 0.4, size=x.size)
X = x.reshape(-1, 1)

eps = 0.5
svr = LinearSVR(epsilon=eps, C=1.0, max_iter=10000, random_state=RND).fit(X, y)
pred = svr.predict(X)
fuera = np.abs(y - pred) > eps      # vectores soporte aproximados
print(f"puntos fuera del tubo (vectores soporte): {fuera.sum()} de {x.size}")

order = np.argsort(x)
fig, ax = plt.subplots(figsize=(7, 5))
ax.scatter(x[~fuera], y[~fuera], s=18, c="steelblue", label="dentro del tubo")
ax.scatter(x[fuera], y[fuera], s=30, c="crimson", label="vectores soporte")
ax.plot(x[order], pred[order], "k-", label="prediccion")
ax.plot(x[order], pred[order] + eps, "k--", label="tubo + eps")
ax.plot(x[order], pred[order] - eps, "k--", label="tubo - eps")
ax.set_title(f"LinearSVR con epsilon={eps}"); ax.legend()
plt.tight_layout(); plt.show()
```

Archivos complementarios

- notebook.ipynb